

The Complete C++ Question Bank

100 Questions · 10 Topics · Beginner → Advanced

Logic Building · Code Writing · Interview Ready

100

Questions

10

Topics

3

Levels

∞

Potential

How to Use This Book

- **Work in order.** Topics are sequenced — each builds on the previous. Don't skip ahead until you've solved the basics.
- **Read before you code.** Each question has a LOGIC INSIGHT — read it before and after solving. Understanding why matters more than getting it to compile.
- **Use the HINT wisely.** Try for at least 20 minutes first. The hint should unstick you, not replace your thinking.
- **Three passes.** Pass 1: Solve it. Pass 2: Optimize it. Pass 3: Explain it out loud as if teaching someone.
- **Levels as checkpoints.** BEGINNER = syntax and concepts. INTERMEDIATE = combining ideas. ADVANCED = real-world complexity.
- **Build a project.** After each topic, build one small project that uses everything from that section.

Topics Overview

NO.	TOPIC	DESCRIPTION	QS
01	Basics & Syntax	Master the foundation — structure, I/O, variables, and operators.	10
02	Control Flow	if/else, switch, loops — how programs make decisions and repeat tasks.	10
03	Functions	Reusable logic blocks — parameters, return values, overloading, recursion.	10
04	Arrays & Strings	Working with collections and text — arrays, 2D arrays, C-strings, and std::string.	10
05	Pointers & Memory	C++'s superpower — direct memory access, dynamic allocation, pointer arithmetic.	10
06	OOP — Classes & Objects	Object-Oriented Programming: classes, encapsulation, constructors, and operator overloading.	10
07	Inheritance & Polymorphism	Building class hierarchies — inheritance, virtual functions, abstract classes, and dynamic dispatch.	10
08	STL & Modern C++	The Standard Template Library + C++11/14/17 features: smart pointers, lambdas, range-for, auto.	10
09	File I/O & Exception Handling	Reading/writing files and building robust programs that handle errors gracefully.	10

10	Data Structures & Algorithms	Linked lists, stacks, queues, trees, graphs — implementing and applying classic CS structures.	10
----	---	--	----

Basics & Syntax

Master the foundation — structure, I/O, variables, and operators.

Q01 **BEGINNER** #I/O #syntax #main()

Write a C++ program that prints 'Hello, World!' to the console and explain every line.

HINT

Use `#include` and `std::cout`

LOGIC INSIGHT — Understand how a C++ program is structured: preprocessor → main function → statements → return.

Q02 **BEGINNER** #data types #variables

Declare variables of types `int`, `float`, `double`, `char`, `bool`, and `string`. Print each with its type name.

HINT

Include for `std::string`

LOGIC INSIGHT — C++ is strongly typed — each variable has a fixed type at compile time.

Q03 **BEGINNER** #operators #cin #arithmetic

Write a program that takes two integers from the user and prints their sum, difference, product, quotient, and remainder.

HINT

Use `cin` for input, careful with integer division

LOGIC INSIGHT — Integer division truncates — $7/2 = 3$, not 3.5 . Use `float/double` for decimal results.

Q04 **BEGINNER** #operators #boolean

Demonstrate all comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) and logical operators (`&&`, `||`, `!`) with examples.

HINT

Print `true/false` results for each expression

LOGIC INSIGHT — These return `bool` values. Understand short-circuit evaluation in `&&` and `||`.

Q05 **BEGINNER** #variables #XOR #swap

Write a program that swaps two variables: (a) using a temp variable, (b) without a temp variable using arithmetic, (c) using XOR.

HINT

```
XOR swap: a^=b; b^=a; a^=b;
```

LOGIC INSIGHT — Three classic swap techniques — good intro to bitwise thinking.

Q06 **INTERMEDIATE** #increment #expressions

Explain and demonstrate the difference between prefix (++i) and postfix (i++) increment operators with output.

HINT

```
int a = ++i returns incremented value; int a = i++ returns original
```

LOGIC INSIGHT — Prefix returns the new value; postfix returns a copy of the old value then increments.

Q07 **BEGINNER** #math #formulas #float

Write a program that reads a temperature in Celsius and converts it to Fahrenheit and Kelvin using the correct formulas.

HINT

```
F = (C * 9/5) + 32; K = C + 273.15
```

LOGIC INSIGHT — Practice type casting: integer arithmetic vs. floating-point arithmetic.

Q08 **INTERMEDIATE** #data types #sizeof #limits

What is the difference between int, long, long long, short? Write a program showing their sizes and max values.

HINT

```
Use sizeof() and for INT_MAX, LONG_MAX
```

LOGIC INSIGHT — Memory size determines the range of values a type can store. $INT_MAX = 2^{31} - 1$.

Q09

INTERMEDIATE

#type casting #narrowing

Demonstrate implicit and explicit type casting. Show what happens when you assign a double to an int (data loss).

HINT

```
Implicit: int x = 3.7; Explicit: int x = (int)3.7;
```

LOGIC INSIGHT — Implicit cast can silently lose data. Explicit cast makes your intent clear. Always be aware.

Q10

INTERMEDIATE

#ternary #expressions #logic

Write a program using the ternary operator to find the largest of three numbers in a single expression.

HINT

```
result = (a>b) ? ((a>c)?a:c) : ((b>c)?b:c)
```

LOGIC INSIGHT — The ternary ? : is a compact if-else for expressions. Nested ternaries work but hurt readability.

TOPIC 02

Control Flow

if/else, switch, loops — how programs make decisions and repeat tasks.

Q01 **BEGINNER** #if-else #conditions

Write a program that reads a number and prints whether it is positive, negative, or zero using if-else-if.

HINT

Three branches: `>0`, `<0`, `==0`

LOGIC INSIGHT — Linear branching with if-else if-else. Only one block executes.

Q02 **BEGINNER** #switch #calculator

Build a simple calculator using switch-case that supports `+`, `-`, `*`, `/` operations on two numbers.

HINT

`switch(op)` with char cases, handle division by zero

LOGIC INSIGHT — switch matches a value to cases. Always add break; to prevent fall-through (unless intentional).

Q03 **INTERMEDIATE** #for loop #primes #nested loops

Print all prime numbers between 1 and 100 using a nested for loop and explain your primality check logic.

HINT

For each `n`, check divisibility up to `sqrt(n)`

LOGIC INSIGHT — Optimization: only check divisors up to `sqrt(n)` — if `n` has a factor `> sqrt(n)`, it also has one `< sqrt(n)`.

Q04 **INTERMEDIATE** #do-while #input validation

Write a do-while loop that keeps asking the user for a password until they enter the correct one. Limit to 3 attempts.

HINT

Use a counter and break when correct or limit reached

LOGIC INSIGHT — do-while executes the body at least once — perfect for menus and input validation.

Q05

BEGINNER

#nested loops #patterns

Print the following pattern using nested loops: * ** *** **** *****

HINT

```
Outer loop: rows; inner loop: print '*' i times
```

LOGIC INSIGHT — Nested loops build 2D patterns. Outer = rows, inner = columns. Think in terms of coordinates.

Q06

INTERMEDIATE

#break #continue

Demonstrate the use of break and continue. Print numbers 1-20 but skip multiples of 3 and stop at the first multiple of 17.

HINT

```
continue skips the rest of iteration; break exits the loop
```

LOGIC INSIGHT — break exits the nearest enclosing loop. continue jumps to the next iteration. Use sparingly for clarity.

Q07

INTERMEDIATE

#while loop #digits #math

Write a program to find the sum of digits of a number using a while loop (e.g., 1234 → 1+2+3+4 = 10).

HINT

```
sum += n % 10; n /= 10; repeat while n > 0
```

LOGIC INSIGHT — % 10 extracts the last digit. / 10 removes it. Classic digit-extraction pattern.

Q08

BEGINNER

#conditions #modulo

Implement FizzBuzz: print 1-100, replace multiples of 3 with 'Fizz', 5 with 'Buzz', both with 'FizzBuzz'.

HINT

```
Check 15 first (both), then 3, then 5
```

LOGIC INSIGHT — Classic interview problem. Order of conditions matters — check the combined case first.

Q09**INTERMEDIATE**

#nested loops #formatting #iomani

Write nested loops to generate the multiplication table (1-10 × 1-10) formatted in aligned columns.

HINT

Use setw(4) from for alignment

*LOGIC INSIGHT — setw() sets minimum width for next output — great for tabular alignment.***Q10****INTERMEDIATE**

#for loop #math #reverse

Use a for loop to reverse a number (e.g., 12345 → 54321) without converting it to a string.

HINT`reversed = reversed * 10 + n % 10; n /= 10;`*LOGIC INSIGHT — Build the reversed number digit by digit by shifting and appending — pure arithmetic reversal.*

Functions

Reusable logic blocks — parameters, return values, overloading, recursion.

Q01 **BEGINNER** #functions #bool return #primes

Write a function `isPrime(int n)` that returns true if `n` is prime. Use it to print all primes up to 200.

HINT

Return bool; check from 2 to `sqrt(n)`

LOGIC INSIGHT — Functions encapsulate logic. A clean `isPrime()` is reusable — don't repeat prime logic everywhere.

Q02 **INTERMEDIATE** #overloading #polymorphism

Write an overloaded function `area()` that calculates: area of a circle (1 arg), rectangle (2 args), triangle (3 args).

HINT

Same function name, different parameter lists

LOGIC INSIGHT — Function overloading: same name, different signatures. Compiler picks the right one at compile time.

Q03 **INTERMEDIATE** #recursion #fibonacci #stack

Implement a recursive function to compute the `n`th Fibonacci number. Compare with iterative version and discuss stack overflow risk.

HINT

`fib(n) = fib(n-1) + fib(n-2); base: fib(0)=0, fib(1)=1`

LOGIC INSIGHT — Recursive fib is $O(2^n)$ — exponential. Iterative is $O(n)$. Recursion has call stack overhead.

Q04 **INTERMEDIATE** #pass by value #pass by reference #&

Demonstrate pass-by-value vs pass-by-reference. Write swap() functions both ways and show which actually modifies the originals.

HINT

```
By-ref: void swap(int &a;, int &b;)
```

LOGIC INSIGHT — Pass-by-value copies — original untouched. Pass-by-reference passes the actual variable — changes persist.

Q05 **INTERMEDIATE** #default params #power

Write a function with default parameters that computes power(base, exponent=2, modulus=0). If modulus > 0, return base^exp % mod.

HINT

Default params go at the end of the parameter list

LOGIC INSIGHT — Default parameters reduce the need for overloads. They must be rightmost in the parameter list.

Q06 **ADVANCED** #recursion #Tower of Hanoi

Write a recursive function to solve the Tower of Hanoi problem for n disks. Print each move.

HINT

Move n-1 disks to auxiliary, move largest to destination, move n-1 back

LOGIC INSIGHT — Classic recursion: the trick is trusting the recursive call handles n-1 correctly.

Q07 **INTERMEDIATE** #arrays #output params #references

Create a function that takes a C-style array and its size, returns the min, max, and average (use output parameters via references).

HINT

```
void stats(int arr[], int n, int &mn;, int &mx;, double &avg;)
```

LOGIC INSIGHT — Multiple return values via reference parameters — a common C-style pattern before std::tuple.

Q08**INTERMEDIATE**

#inline #optimization

Write an inline function for absolute value. Explain when inline is beneficial and when it's not.

HINT

```
inline int abs_val(int x) { return x < 0 ? -x : x; }
```

LOGIC INSIGHT — Inline replaces the function call with its body — eliminates call overhead for tiny functions.

Q09**INTERMEDIATE**

#static variables #state

Implement a function that uses static local variables to count how many times it has been called, without global variables.

HINT

```
static int count = 0; inside the function body
```

LOGIC INSIGHT — Static local vars are initialized once and persist across calls — local scope but program lifetime.

Q10**ADVANCED**

#templates #generics

Write a function template `max_val` that returns the larger of two values. Test it with `int`, `double`, and `char`.

HINT

```
template T max_val(T a, T b)
```

LOGIC INSIGHT — Function templates generate code for each type used — one definition, many instantiations.

TOPIC 04

Arrays & Strings

Working with collections and text — arrays, 2D arrays, C-strings, and std::string.

Q01 **BEGINNER** #arrays #indexing #reverse

Declare an array of 10 integers, fill it with user input, then print it in reverse order.

HINT

Iterate from index `n-1` down to `0`

LOGIC INSIGHT — Arrays are zero-indexed. Reverse iteration: start at size-1, go to 0.

Q02 **INTERMEDIATE** #sorting #bubble sort #nested loops

Implement bubble sort on an integer array. Print the array after each pass to show the progression.

HINT

Compare adjacent elements; outer loop `n` times, inner loop `n-i-1`

LOGIC INSIGHT — Bubble sort: largest elements 'bubble' to the end each pass. $O(n^2)$ — simple but slow.

Q03 **INTERMEDIATE** #binary search #sorted arrays

Write binary search on a sorted array. Return the index of a target, or -1 if not found.

HINT

`mid = (low + high) / 2;` compare and adjust low or high

*LOGIC INSIGHT — Binary search: $O(\log n)$. Halves the search space each step. **REQUIRES** sorted array.*

Q04 **INTERMEDIATE** #2D arrays #matrix #transpose

Create a 3x3 2D array (matrix), take user input, and compute the transpose (swap rows and columns).

HINT

Transpose: `result[j][i] = original[i][j]`

LOGIC INSIGHT — 2D arrays: `arr[row][col]`. Transposing swaps row and column indices.

Q05 **INTERMEDIATE** #std::string #palindrome #char functions

Using `std::string`: reverse it, count vowels, check if it's a palindrome — all in separate functions.

HINT

Use `string.length()`, `string[i]`, `tolower()` for case-insensitive check

LOGIC INSIGHT — `std::string` is a class with methods. Treat it like a smart char array with bounds checking.

Q06 **INTERMEDIATE** #character arrays #frequency #ASCII

Write a program that counts the frequency of each character in a string using an array of size 26 (for a-z).

HINT

`freq[ch - 'a']++` for lowercase chars

LOGIC INSIGHT — ASCII trick: 'a' = 97, so `ch - 'a'` maps 'a'→0, 'b'→1 ... 'z'→25. Elegant indexing.

Q07 **INTERMEDIATE** #C-strings #null terminator #manual implementation

Implement your own versions of `strlen()`, `strcpy()`, and `strcat()` for C-style strings without using `.`

HINT

C-strings are null-terminated char arrays. Loop until `'\0'`

LOGIC INSIGHT — C-strings end at `'\0'`. Understanding this reveals why buffer overflows are dangerous.

Q08 **INTERMEDIATE** #string manipulation #toupper #words

Write a function that takes a sentence and returns it with each word's first letter capitalized (title case).

HINT

Capitalize after `' '` or at position 0. Use `toupper()`

LOGIC INSIGHT — Iterate char by char; state tracking (was-previous-a-space) drives capitalization.

Q09 **ADVANCED** #matrix multiplication #2D arrays # $O(n^3)$

Implement matrix multiplication for two 3x3 matrices. Explain the time complexity.

HINT

`result[i][j] += a[i][k] * b[k][j]` — triple nested loop

LOGIC INSIGHT — Matrix multiply: $O(n^3)$ for $n \times n$. The k -loop accumulates the dot product of row i and col j .

Q10

ADVANCED

#anagram #frequency array #O(n)

Write a function that checks if two strings are anagrams (same chars, different order) in $O(n)$ time.

HINT

Sort both and compare, or use a frequency array of 26 ints

LOGIC INSIGHT — Frequency array approach: increment for string1, decrement for string2. All zeros = anagram.

TOPIC 05

Pointers & Memory

C++'s superpower — direct memory access, dynamic allocation, pointer arithmetic.

Q01 **BEGINNER** #pointers #& #*

Declare a pointer to an int, assign it an address using &, print the value and the address. Explain * and & operators.

HINT

```
int x = 10; int* ptr = &x; *ptr = 20;
```

*LOGIC INSIGHT — & = address-of operator. * = dereference (get value at address). Pointer = variable that stores an address.*

Q02 **BEGINNER** #pointers #swap #dereference

Write a swap function using pointers (not references). Demonstrate that the original values change.

HINT

```
void swap(int* a, int* b) — use *a and *b to access values
```

LOGIC INSIGHT — Passing a pointer gives the function access to the original variable via its address.

Q03 **INTERMEDIATE** #dynamic memory #new #delete

Use new and delete to dynamically allocate an integer array of size n (user-input). Fill it, print it, then free the memory.

HINT

```
int* arr = new int[n]; ... delete[] arr;
```

LOGIC INSIGHT — new allocates on the heap. delete/delete[] must match new/new[]. Forgetting = memory leak.

Q04 **INTERMEDIATE** #pointer arithmetic #arrays #traversal

Demonstrate pointer arithmetic: declare an array, use a pointer to traverse it (ptr++), and show pointer subtraction.

HINT

```
int* ptr = arr; *(ptr+2) is arr[2]; ptr++ advances by sizeof(int) bytes
```

LOGIC INSIGHT — ptr++ advances by the size of the pointed-to type — not just 1 byte. Arrays and pointers are deeply connected.

Q05 **ADVANCED** #const pointer #pointer to const

Explain and demonstrate the difference between const int* ptr, int* const ptr, and const int* const ptr.

HINT

Read right-to-left: 'ptr is a const pointer to int' vs 'ptr is a pointer to const int'

LOGIC INSIGHT — const placement changes what's immutable: the pointer itself, or the data it points to, or both.

Q06 **ADVANCED** #dynamic strings #ownership #memory management

Write a function that returns a dynamically allocated string (char*). The caller must free it. Demonstrate proper usage.

HINT

```
Allocate with new char[len+1], copy, return pointer. Caller calls delete[]
```

LOGIC INSIGHT — Ownership principle: whoever allocates is responsible for freeing. This is the root of many bugs.

Q07 **ADVANCED** #function pointers #callbacks

Create a pointer to a function: write two functions (add, multiply) and call them through a function pointer.

HINT

```
int (*funcPtr)(int, int) = &add; funcPtr(3, 4);
```

LOGIC INSIGHT — Function pointers enable callbacks and runtime function selection — foundational for C-style polymorphism.

Q08**ADVANCED**

#linked list #stack #dynamic allocation

Implement a dynamic stack (push/pop) using a pointer-based linked list without STL.

HINT

```
struct Node { int val; Node* next; }; maintain a top pointer
```

*LOGIC INSIGHT — Each node contains data + pointer to next. Push = prepend new node. Pop = remove head.***Q09****ADVANCED**

#dangling pointer #memory leak #bugs

Explain what a dangling pointer and memory leak are. Write code that creates both, then fix them.

HINT

Dangling: delete ptr then use *ptr. Leak: allocate, go out of scope without delete

*LOGIC INSIGHT — Dangling pointer = undefined behavior (crash). Memory leak = resource not returned to OS.***Q10****ADVANCED**

#void pointer #reinterpret_cast #generic

Use a void* pointer as a generic container. Store an int and a double, cast back with reinterpret_cast and print.

HINT

```
void* vp = &x; int* ip = reinterpret_cast(vp);
```

LOGIC INSIGHT — void loses type info — you must cast back to the correct type. Wrong cast = undefined behavior.*

OOP — Classes & Objects

Object-Oriented Programming: classes, encapsulation, constructors, and operator overloading.

Q01 **BEGINNER** #classes #encapsulation #private/public

Design a BankAccount class with private balance, public deposit(), withdraw(), and getBalance(). Prevent negative balance on withdrawal.

HINT

Use private: for data, public: for interface. Validate in withdraw()

LOGIC INSIGHT — Encapsulation: hide data, expose controlled interface. The class enforces its own invariants.

Q02 **INTERMEDIATE** #constructor #destructor #copy constructor

Add a constructor, copy constructor, and destructor to the BankAccount class. Print messages in each to trace their calls.

HINT

BankAccount(double b); BankAccount(const BankAccount& other); ~BankAccount();

LOGIC INSIGHT — The Big Three: if you define any of constructor/copy-ctor/destructor, define all three.

Q03 **INTERMEDIATE** #operator overloading #+ #Vector2D

Overload the + operator for a Vector2D class (x, y components) so that v1 + v2 returns a new vector.

HINT

Vector2D operator+(const Vector2D& other) const;

LOGIC INSIGHT — Operator overloading lets custom types behave like built-ins. Keep semantics intuitive.

Q04 **INTERMEDIATE** #<< #== #friend functions

Create a Rectangle class. Overload << to print it, and overload == to check if two rectangles have equal area.

HINT

friend ostream& operator<<(ostream& os, const Rectangle& r);

LOGIC INSIGHT — friend allows non-member functions to access private members — needed for << and >> overloads.

Q05 **INTERMEDIATE** #static member #object count

Implement a Student class with a static variable that counts the total number of Student objects created.

HINT

```
static int count; – increment in constructor, decrement in destructor
```

LOGIC INSIGHT — Static members belong to the class, not instances. Shared across all objects.

Q06 **ADVANCED** #matrix class #operator+ #operator*

Write a class Matrix that supports addition and multiplication via overloaded operators. Include bounds checking.

HINT

Store data in a 2D vector. Throw exception if dimensions don't match.

LOGIC INSIGHT — Good class design: enforce preconditions (dimension compatibility) at the point of use.

Q07 **ADVANCED** #class templates #generics #stack

Implement a generic Stack class template with push(), pop(), peek(), and isEmpty() methods.

HINT

```
template class Stack { private: vector data; ... };
```

LOGIC INSIGHT — Class templates are blueprints. Stack and Stack are distinct types generated from one template.

Q08 **ADVANCED** #Rule of Three #deep copy #operator=

Demonstrate the Rule of Three: build a MyString class with raw char* data that correctly handles copy construction and assignment.

HINT

Deep copy in copy ctor and operator=. Free old memory in operator= before copying.

LOGIC INSIGHT — Default copy = shallow copy (shared pointer). Deep copy = independent duplicate. Critical for classes owning heap memory.

Q09**ADVANCED**

#Fraction #GCD #full operator overloading

Create a Fraction class with numerator and denominator. Overload +, -, *, /, ==, and <= with automatic reduction to lowest terms.

HINT

Use GCD to simplify. gcd(a,b) from in C++17

LOGIC INSIGHT — Full operator overloading exercise — great practice for thinking about class invariants.

Q10**ADVANCED**

#Singleton #design pattern #static

Implement a Singleton pattern: a class where only one instance can exist. Provide a static getInstance() method.

HINT

Private constructor; static MyClass* instance; static getInstance() creates if null

LOGIC INSIGHT — Singleton ensures a single shared instance. Constructor is private — only the class itself can create it.

Inheritance & Polymorphism

Building class hierarchies — inheritance, virtual functions, abstract classes, and dynamic dispatch.

Q01 **BEGINNER** #inheritance #virtual #override

Create a base class **Animal** with a **speak()** method. Derive **Dog** and **Cat** from it. Override **speak()** in each. Call via **Animal** pointer.

HINT

```
virtual void speak() in base; override in derived. Animal* a = new Dog();  
a->speak();
```

LOGIC INSIGHT — Virtual functions enable dynamic dispatch — the correct override is called based on the actual object type, not the pointer type.

Q02 **INTERMEDIATE** #constructor order #initializer list

Demonstrate constructor chaining: when a **Derived** object is created, show the order of **Base** and **Derived** constructors/destructors.

HINT

```
Add print statements. Use : Base(args) in derived constructor initializer  
list.
```

LOGIC INSIGHT — Base constructor always runs first. Destructors run in reverse order: Derived first, then Base.

Q03 **INTERMEDIATE** #abstract class #pure virtual #polymorphism

Create an abstract **Shape** class with a pure virtual **area()** method. Derive **Circle**, **Rectangle**, **Square** from it. Use **Shape*** array.

HINT

```
virtual double area() = 0; makes the function pure virtual and Shape abstract
```

LOGIC INSIGHT — Pure virtual functions create abstract base classes — they can't be instantiated but enforce interface contracts on derived classes.

Q04 **INTERMEDIATE** #virtual #hiding #slicing

Explain the difference between method overriding (virtual) and method hiding (non-virtual). Show both with same function name.

HINT

```
Non-virtual: Base* b = new Derived(); b->func() calls Base::func() – no dispatch
```

LOGIC INSIGHT — Without virtual, call is resolved at compile time (static dispatch). With virtual, at runtime (dynamic dispatch).

Q05 **ADVANCED** #multiple inheritance #diamond problem #virtual base

Implement multiple inheritance: class FlyingFish inherits from both Fish and Bird. Handle the diamond problem using virtual inheritance.

HINT

```
class Fish : virtual public Animal {}; class Bird : virtual public Animal {};
```

LOGIC INSIGHT — Diamond problem: two paths to the same base class. Virtual inheritance ensures only one base subobject.

Q06 **ADVANCED** #virtual destructor #memory leak #polymorphism

Why must the base class destructor be virtual when using polymorphism? Demonstrate the bug without it and the fix with it.

HINT

```
Without virtual destructor, delete on base pointer only calls Base::~~Base() – derived part leaks
```

LOGIC INSIGHT — A non-virtual destructor on a base class causes UB when deleting via base pointer. Always make destructor virtual in polymorphic base classes.

Q07 **INTERMEDIATE** #polymorphism #vector #hierarchy

Implement an Employee hierarchy: Employee → Manager, Developer. Each has calcSalary(). Use a vector to print all salaries.

HINT

```
virtual double calcSalary() in Employee. Manager might get a bonus multiplier.
```

LOGIC INSIGHT — Storing derived objects as base pointers in a container is idiomatic C++ polymorphism.

Q08 **ADVANCED** #dynamic_cast #RTTI #downcasting

Use `dynamic_cast` to safely downcast from base to derived pointer. Handle failure when the cast is invalid.

HINT

```
Derived* d = dynamic_cast<Derived*>(base_ptr); if(d == nullptr) - cast failed
```

LOGIC INSIGHT — `dynamic_cast` checks type at runtime (RTTI). Returns `nullptr` on failure — safer than `static_cast`.

Q09 **ADVANCED** #Strategy pattern #OOP design

Implement the Strategy pattern using virtual functions: a `Sorter` class whose sorting algorithm can be swapped at runtime.

HINT

Abstract `SortStrategy` with `sort(vector&) = 0`. Concrete `BubbleSort`, `QuickSort` classes.

LOGIC INSIGHT — Strategy pattern: encapsulate algorithms behind an interface. Swap behavior without changing the client.

Q10 **ADVANCED** #Visitor pattern #double dispatch

Write a class hierarchy for a simple Shape renderer that uses the Visitor pattern to add `area()` and `perimeter()` without modifying Shape classes.

HINT

```
class Visitor { virtual void visit(Circle&) = 0; virtual void visit(Rect&) = 0; };
```

LOGIC INSIGHT — Visitor adds operations to a class hierarchy without modifying it — open/closed principle.

TOPIC 08

STL & Modern C++

The Standard Template Library + C++11/14/17 features: smart pointers, lambdas, range-for, auto.

Q01 **BEGINNER** #vector #STL #range-for

Use `std::vector` to build a dynamic list of student names. Push, pop, insert at index, erase, and iterate using range-for.

HINT

```
v.push_back(), v.pop_back(), v.insert(v.begin()+i, val), v.erase(v.begin()+i)
```

LOGIC INSIGHT — `vector` is the go-to dynamic array in C++. It manages its own memory and resizes automatically.

Q02 **INTERMEDIATE** #map #word frequency #iteration

Use `std::map` to count the frequency of each word in a paragraph. Print them sorted alphabetically.

HINT

```
map freq; freq[word]++; maps are sorted by key by default
```

LOGIC INSIGHT — `map` is a sorted associative container. `operator[]` creates a zero-initialized entry if key is absent.

Q03 **INTERMEDIATE** #sort #lambda #comparator

Demonstrate `std::sort` with a custom comparator. Sort a vector of structs by multiple fields (first by score descending, then by name ascending).

HINT

```
std::sort(v.begin(), v.end(), [](const S& a, const S& b){ return a.score !=  
b.score ? a.score > b.score : a.name < b.name; });
```

LOGIC INSIGHT — Custom comparators give you complete control over ordering. Lambda syntax: `[captures](params) { body }`.

Q04 **INTERMEDIATE** #unique_ptr #smart pointers #move semantics

Use `std::unique_ptr` to manage a dynamically allocated object. Demonstrate ownership transfer with `std::move`.

HINT

```
auto p = std::make_unique(); auto p2 = std::move(p); // p is now null
```

LOGIC INSIGHT — `unique_ptr`: sole owner, auto-deletes when out of scope. `std::move` transfers ownership — original becomes null.

Q05 **ADVANCED** #shared_ptr #weak_ptr #reference cycle

Use `std::shared_ptr` and `std::weak_ptr` together. Create a scenario where `weak_ptr` breaks a reference cycle.

HINT

Two objects pointing to each other with `shared_ptr` = cycle. One should use `weak_ptr` instead.

LOGIC INSIGHT — `shared_ptr` uses reference counting. Cycle = count never reaches 0 = leak. `weak_ptr` doesn't increase count.

Q06 **INTERMEDIATE** #algorithms #iterators #functional

Write a program using `std::algorithm` functions: `find_if`, `count_if`, `transform`, `accumulate`, `remove_if` on a vector of integers.

HINT

#include and . Each takes begin/end iterators + predicate/function

LOGIC INSIGHT — STL algorithms + iterators = generic, reusable code. Learn to think in terms of ranges.

Q07 **INTERMEDIATE** #structured bindings #tuple #C++17

Use structured bindings (C++17) and `std::pair`/`std::tuple` to return and destructure multiple values from a function.

HINT

```
auto [min, max] = findMinMax(v); return std::make_pair(min, max);
```

LOGIC INSIGHT — Structured bindings unpack tuple/pair/struct into named variables — much more readable than `get<0>()`.

Q08**ADVANCED**

#std::function #observer pattern #callbacks

Implement a simple observer pattern using `std::function` and `std::vector` to store callbacks. Fire all observers on an event.

HINT

```
vector< std::function<void()> > observers; for(auto& cb : observers) cb(value);
```

LOGIC INSIGHT — `std::function` wraps any callable (function, lambda, functor). Enables flexible callback systems.

Q09**ADVANCED**

#unordered_map #hash table #performance

Use `std::unordered_map` to implement an $O(1)$ average-case lookup cache. Contrast performance with `std::map`.

HINT

`unordered_map` uses hash table internally. No ordering guarantee but $O(1)$ avg lookup.

LOGIC INSIGHT — `map` = $O(\log n)$ sorted tree. `unordered_map` = $O(1)$ average hash table. Choose based on need: ordering vs speed.

Q10**ADVANCED**

#ranges #views #C++20

Write a pipeline using `std::ranges` (C++20) or `std::transform` + `std::copy_if` to filter even numbers from a list and double them.

HINT

```
C++20: auto result = nums | views::filter(isEven) |  
views::transform(double_it);
```

LOGIC INSIGHT — Ranges are lazy: they compute only what's needed. Composable pipelines are readable and efficient.

File I/O & Exception Handling

Reading/writing files and building robust programs that handle errors gracefully.

Q01 **BEGINNER** #ofstream #ifstream #CSV

Write a program that writes student names and grades to a CSV file, then reads it back and prints a formatted table.

HINT

ofstream to write, ifstream to read. Use getline() and stringstream to parse CSV.

LOGIC INSIGHT — File streams work like cin/cout but connected to files. Always check is_open() and close() when done.

Q02 **INTERMEDIATE** #file reading #word count #set

Write a word count utility that reads any text file and reports: total characters, words, lines, and unique words.

HINT

getline() for lines, >> for words. Use set for unique words.

LOGIC INSIGHT — Combine file I/O with data structures. set automatically deduplicates.

Q03 **BEGINNER** #try-catch #throw #runtime_error

Demonstrate try-catch-throw. Write a function safeDivide() that throws a std::runtime_error when dividing by zero.

HINT

```
throw runtime_error('division by zero'); catch(const exception& e) { cout << e.what(); }
```

LOGIC INSIGHT — Exceptions separate error handling from normal flow. Always catch by const reference to avoid slicing.

Q04 **INTERMEDIATE** #custom exception #exception hierarchy

Create a custom exception class `InsufficientFundsException` derived from `std::exception`. Use it in a `BankAccount`.

HINT

```
class InsufficientFundsException : public std::exception { const char* what() const noexcept override {...} };
```

LOGIC INSIGHT — Custom exceptions carry domain-specific info. Override `what()` to return a meaningful error message.

Q05 **INTERMEDIATE** #file copy #RAII #exception safety

Implement exception-safe file copying: read source, write to destination. Handle file-not-found, permission errors. Clean up on any failure.

HINT

Use RAII — file streams close automatically. Wrap operations in try-catch.

LOGIC INSIGHT — RAII (Resource Acquisition Is Initialization): tie resource lifetime to object lifetime — destructors clean up automatically.

Q06 **ADVANCED** #binary I/O #read() #write()

Write a program that reads a binary file containing integers (written in binary mode), and prints their sum.

HINT

`ios::binary` mode. Use `read()` and `write()` instead of `<<` `>>`. `reinterpret_cast` for raw bytes.

LOGIC INSIGHT — Binary I/O is faster and compact — stores exact memory representation. Text I/O is human-readable but larger.

Q07 **INTERMEDIATE** #logger #ios::app #timestamps

Implement a simple logger class that writes timestamped messages to a log file. Support log levels: INFO, WARNING, ERROR.

HINT

Use for timestamps. `ofstream` in append mode (`ios::app`).

LOGIC INSIGHT — `ios::app` opens in append mode — won't overwrite existing log entries.

Q08**INTERMEDIATE**

#multiple catch #exception hierarchy

Demonstrate multiple exception types: write a function that can throw different exceptions based on input. Catch each with separate catch blocks.

HINT

```
catch(const out_of_range& e) catch(const invalid_argument& e) catch(const exception& e) – order from specific to general
```

LOGIC INSIGHT — Catch blocks are checked in order — put specific types before general. catch(exception&) is the catch-all.

Q09**ADVANCED**

#fseek #ftell #file position

Use fseek() / ftell() with C-style FILE* to implement a function that returns the file size without reading all bytes.

HINT

```
fseek(f, 0, SEEK_END); long size = ftell(f); rewind(f);
```

LOGIC INSIGHT — SEEK_END moves to the end. ftell() returns the current position = file size in bytes.

Q10**ADVANCED**

#serialization #binary I/O #persistence

Build a simple serialization system: a class that can write its state to a file (serialize) and read it back (deserialize).

HINT

Write each member to file in a defined order. Read in the same order. Use binary mode for efficiency.

LOGIC INSIGHT — Serialization = converting object state to bytes. Deserialization = bytes back to object. Order and types must match exactly.

Data Structures & Algorithms

Linked lists, stacks, queues, trees, graphs — implementing and applying classic CS structures.

Q01 **INTERMEDIATE** #linked list #nodes #pointers

Implement a singly linked list with `append()`, `prepend()`, `delete(val)`, and `print()` operations from scratch.

HINT

```
struct Node { int data; Node* next; }; Track head pointer.
```

LOGIC INSIGHT — Linked list: each node knows only the next. No random access — only sequential. Insertion/deletion $O(1)$ at known position.

Q02 **INTERMEDIATE** #stack #array-based

Implement a stack using arrays (fixed size) with `push`, `pop`, `peek`, `isFull`, `isEmpty`. Handle overflow and underflow.

HINT

```
int data[MAX]; int top = -1; push increments top, pop decrements.
```

LOGIC INSIGHT — Stack: LIFO (Last In First Out). Top pointer tracks the current top. -1 = empty. MAX-1 = full.

Q03 **ADVANCED** #queue #circular array

Implement a queue using a circular array to avoid wasted space after dequeues.

HINT

```
front = (front + 1) % MAX; rear = (rear + 1) % MAX; count tracks size
```

LOGIC INSIGHT — Circular buffer: front/rear wrap around using modulo. Avoids shifting and maximizes fixed memory use.

Q04 **INTERMEDIATE** #BST #tree traversal #recursion

Build a Binary Search Tree: `insert()`, `search()`, and `inOrder()` traversal. Show that `inOrder` gives sorted output.

HINT

```
struct BSTNode { int val; BSTNode *left, *right; }; Recursively insert/search.
```

LOGIC INSIGHT — BST property: $left < node < right$. InOrder (left→root→right) produces sorted output. $O(\log n)$ for balanced trees.

Q05 **ADVANCED** #merge sort # $O(n \log n)$ #divide and conquer

Implement merge sort. Analyze its time complexity and explain why it's preferred over bubble sort for large datasets.

HINT

Divide array in half recursively. Merge two sorted halves. $O(n \log n)$.

LOGIC INSIGHT — Merge sort: $O(n \log n)$ always — doesn't degrade to $O(n^2)$. Divide and conquer paradigm.

Q06 **ADVANCED** #hash table #separate chaining #collisions

Implement a hash table from scratch with separate chaining. Support insert, search, and delete operations.

HINT

array of linked lists. $\text{hash}(\text{key}) = \text{key} \% \text{TABLE_SIZE}$. Chain collisions in the list.

LOGIC INSIGHT — Hash function maps key to index. Collisions resolved by chaining (linked list at each bucket).

Q07 **ADVANCED** #Dijkstra #shortest path #graph

Write Dijkstra's shortest path algorithm on a weighted graph represented as an adjacency matrix.

HINT

Use a `visited[]` array and `dist[]`. Greedily pick the unvisited node with smallest `dist`.

LOGIC INSIGHT — Dijkstra: greedy algorithm. Always extend the cheapest unvisited node. Only works with non-negative weights.

Q08 **ADVANCED** #heap #heapify #heapsort

Implement a min-heap from scratch. Add `heapifyUp` (insert) and `heapifyDown` (remove min). Test with sorting (heapsort).

HINT

Heap stored in array: parent at `i`, children at `2i+1`, `2i+2`. Swap with parent while `child < parent`.

LOGIC INSIGHT — Heap: complete binary tree stored in array. Parent always smaller (min-heap). Insert + `heapify-up`. Remove min + `heapify-down`.

Q09

ADVANCED

#dynamic programming #knapsack #DP table

Solve the 0/1 Knapsack problem using dynamic programming. Build and explain the 2D DP table.

HINT

```
dp[i][w] = max value using first i items with weight capacity w. Either
include item i or skip it.
```

LOGIC INSIGHT — DP: solve subproblems once, store results. Bottom-up builds the table from base cases up.

Q10

ADVANCED

#BFS #DFS #graph traversal

Implement BFS and DFS on a graph (adjacency list). Use BFS to find the shortest path (unweighted). Use DFS to detect cycles.

HINT

```
BFS: use queue. DFS: use stack or recursion. Track visited[] to avoid infinite
loops.
```

LOGIC INSIGHT — BFS explores level by level — guarantees shortest path in unweighted graphs. DFS goes deep first — useful for connectivity and cycle detection.